Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

8. Implement an N-gram language model

Aim: To build an N-gram model that predicts the next word using the preceding \(N-1\) words.

```
import nltk
from nltk.util import ngrams
from collections import Counter

# Sample training sentences.
sentences = [
    "the cat sits on the mat",
    "the cat eats fish",
    "the dog sits on the mat"
]

# Convert each sentence to lowercase tokens and add boundary markers.
tokenized_sentences = [
    ['<s>'] + sentence.lower().split() + ['</s>']
    for sentence in sentences
]

# Choose N = 3, which means a trigram model.
N = 3

# Collect all trigrams from the training corpus.
ngram_counts = Counter()
context_counts = Counter()

for tokens in tokenized_sentences:
    # Generate N-grams.
    generated_ngrams = list(ngrams(tokens, N))

    # Count every N-gram.
    ngram_counts.update(generated_ngrams)

    # Count the context consisting of the first N-1 words.
    for gram in generated_ngrams:
        context = gram[:-1]
        context_counts[context] += 1

# Print the learned trigram frequencies.
print("Trigram Frequencies:")
for gram, count in ngram_counts.items():
    print(gram, ":", count)

# Calculate the probability of a trigram.
context = ('the', 'cat')
next_word = 'sits'

gram = context + (next_word,)

if context_counts[context] > 0:
    probability = ngram_counts[gram] / context_counts[context]
else:
    probability = 0.0

print("\nContext:", context)
print("Next word:", next_word)
print("Probability:", probability)
```

Sample output

```
Trigram Frequencies:
('<s>', 'the', 'cat') : 2
('the', 'cat', 'sits') : 1
('cat', 'sits', 'on') : 1
...

Context: ('the', 'cat')
Next word: sits
Probability: 0.5
```

Explanation:

• Unigram: one word.

• Bigram: two consecutive words.

• Trigram: three consecutive words.

• N-gram: a sequence of \(N\) consecutive tokens.

For a trigram model:

\[

P(w_i\mid w_{i-2},w_{i-1})

=\frac{C(w_{i-2},w_{i-1},w_i)}

{C(w_{i-2},w_{i-1})}

\]

The model uses the preceding two words to estimate the next word's probability.